

OPEN MP

Q1. Write a OpenMP Program with c/c++ that estimates the value of π using a integration formula (rectangle rule).

$$\text{Area} = \int_0^1 \frac{4}{1+x^2} dx = \pi$$

a) write the sequential version program to estimate the value of π (L.A). Test the result with classical integration value.

⇒ Algorithm

1. Start
2. Initialize no of intervals: N .
3. Calculate the width of each rectangle: $w = 1/N$.
4. Initialize sum = 0.0.
5. For interval from 0 to $N-1$:
 - Compute midpoint x .
 - Evaluate $4/(1+x^2)$.
 - Add the value to sum.
6. Compute $\pi = \text{sum} \times w$.
7. Print the value of π .
8. Stop.

b) Parallel π Estimation using, OpenMP.

⇒ Algorithm

1. Start
2. Initialize N , w & sum.
3. Create multiple threads using OpenMP.
4. Divide loop iterations among threads using #pragma omp parallel for.
5. Each thread:
 - Computes midpoint x .
 - Calculate rectangle area.

- Display: the thread ID & contribution.
- Add: the result to the shared sum.
- 6. Combine all thread contributions.
- 7. Compute $P2 = W \times \text{sum}$.
- 8. Print it & execution details.
- 9. Stop.

c) Removing Race Condition

⇒ Algorithm

1. Start
2. Identify the shared variable (sum) causing race condition.
3. Apply OpenMP reduction ($\&: \text{sum}$) clause.
4. Execute the parallel loop.
5. Each thread maintains a private partial sum.
6. Compute it.
7. Compare the result with the sequential version.
8. Stop.

Q2) : False Sharing

- a) create an OpenMP program where each thread updates its own counter stored in a contiguous integer array (false sharing). Print the corresponding execution time.

⇒ Algorithm

1. Start
2. Create an array of counters.
3. Assign one counter to each thread.
4. Each thread repeatedly updates its counter.
5. Measure execution time.
6. Print execution time.
7. Stop.

b) Modify the program using a padded structure so that each counter occupies a separate cache line. Measure the execution time.

⇒ Algorithm

1. Start
2. Create a padded structure so each counter occupies one cache line.
3. Assign one padded counter to each thread.
4. Each thread updates only its own counter.
5. Measure execution time.
6. Print execution time.
7. Stop.

c) Implement an alternative solution using OpenMP reduction & compare the false sharing, padded & reduction versions.

Sol ⇒ Algorithm

1. Start
2. Initialize a shared result variable.
3. Use OpenMP reduction clause.
4. Each thread computes a local value.
5. OpenMP combines local values automatically.
6. Measure execution time.
7. Compare execution times of all three methods.
8. Stop.